AL 2002 - ARTIFICIAL INTELLIGENCE LAB

PROJECT GUIDE

Battle City | Tank 1990

Implementing AI Concepts Through Classic Game Development

Sections	6A & 6B
Semester	Spring 2026
Team Size	**2 persons (at max) or individual**
Language	Python
Levels	2 Regular Levels + 1 Boss Level
Total Tank Types	5 Enemy Types + 1 Boss Tank
Core Modules	CSP | Search | Agents | Adversarial

Creativity is encouraged — the implementation logic is your own!

Prepared by: *Muhammad Ahsan*
Reviewed by: *Daniyal Shafique, Dr. Rabia Maqsood*

For questions/clarification, write an email to both the TAs (f223255@cfd.nu.edu.pk, f223598@cfd.nu.edu.pk) and Cc your lab instructor.

1. How the Game Works — Detailed Explanation

Battle City (Tank 1990) is a top-down 2D grid-based game. Every element — tanks, bullets, walls, and the base — lives on a 26x26 tile grid. Understanding how each system works is essential before you implement any AI.

https://www.retrogames.cz/play_1412-NES.php

1.1 The Grid & Coordinate System

The entire game world is a 26x26 matrix of tiles. Every game object (tank, bullet, wall) is aligned to this grid at all times. There is no sub-tile movement — a tank either occupies a cell or it does not.

- Top-left is (0,0). X increases rightward; Y increases downward.
- The Eagle (Base) is always fixed at the bottom-center of the map: approximately tile (12, 24).
- Enemy spawn points are fixed: Top-Left (0,0), Top-Center (12,0), Top-Right (24,0).
- The player spawns near the bottom-left, around tile (4, 24).

1.2 Movement System

Tanks move one tile at a time in four cardinal directions: Up, Down, Left, Right. There is no diagonal movement. On each game tick, a moving tank attempts to advance one tile in its current direction.

- Before moving, the game checks the destination tile's terrain type.

- If the tile is Empty (0) or Forest (4), the move is allowed.
- If the tile is Brick (1), Steel (2), or Water (3), the move is blocked. The tank stays in place.
- If the tile contains another tank (player or enemy), the move is blocked — tanks are solid objects.
- The player's tank can only move while not shooting — or shoot while stationary. (Implement your own rule here.)

1.3 Shooting & Bullet System

Each tank fires one bullet at a time in the direction it is currently facing. A bullet travels across tiles each game tick until it hits something.

- A bullet's speed is typically 2x the tank movement speed (it crosses tiles faster).
- When a bullet hits a Brick Wall (1): the wall is destroyed — that tile becomes Empty (0). This is a permanent map change.
- When a bullet hits a Steel Wall (2): the bullet is destroyed. The steel wall remains intact.
- When a bullet hits a tank: the tank takes one hit of damage. Basic tanks die in 1 hit; Armor tanks survive 4 hits.
- When two bullets collide mid-air: both bullets are destroyed — neither continues. This is an advanced defensive tactic.
- When a bullet hits the Eagle (5): the game is immediately over — the player loses.
- **Bullets cannot pass through Water (3) or walls. They pass through Forest (4) tiles without being blocked.**

1.4 Game Loop — What Happens Each Tick

The game runs a continuous loop. On each tick, the following sequence executes in order:

1. INPUT: Read player keyboard input (move direction / shoot command).
2. AGENT DECISIONS: Each enemy AI agent runs its decision logic and selects an action (move or shoot).
3. MOVE: All tanks attempt to move to their target tile. Collision checks run against terrain and other tanks.
4. SHOOT: All tanks that chose to shoot this tick fire a bullet.
5. BULLET UPDATE: All active bullets advance one tile in their direction.
6. COLLISION DETECTION: Check bullet vs. wall, bullet vs. tank, bullet vs. bullet, bullet vs. Eagle.
7. STATE UPDATE: Destroy walls, reduce tank HP, remove dead tanks, apply power-up effects.
8. SPAWN CHECK: If fewer than 4 enemies are active and enemies remain in the level pool, spawn one.
9. RENDER: Draw the updated grid, tanks, bullets, UI elements to screen.
10. WIN/LOSE CHECK: If all 20 enemies are destroyed => Win. If player HP = 0 or Eagle destroyed => Lose.

1.5 Destruction & Dynamic Map

One of Battle City's defining features is that the map changes during gameplay. When any brick wall is shot and destroyed, that tile permanently becomes empty for the rest of the level. This has far-reaching AI implications:

- A path that was blocked at the start of the level may open up mid-game.
- The player can deliberately shoot walls to create shortcuts to intercept enemies.
- An AI agent that cached a path must re-validate it every few ticks because the map may have changed.
- The Eagle's surrounding walls can be destroyed, making the base vulnerable — a core strategic concern.

1.6 Spawning System

Each level contains a fixed pool of 20 enemy tanks. They do not all appear at once. Instead, they trickle in through the three spawn points at the top of the map.

- At most 3-4 enemy tanks are active on the map simultaneously.
- When an enemy is destroyed, the next tank from the pool spawns after a short delay.
- Spawn point selection can be: fixed rotation (Left, Center, Right, Left...) or random — your choice.
- Fairness constraint: no enemy may spawn within 10 tiles (Manhattan distance) of the player's current position.

1.7 Lives, HP, and Win/Lose Conditions

The player starts with 10 lives and loses one life each time their tank is destroyed.

- Basic enemy tanks: 1 hit to destroy.
- Fast tanks: 1 hit to destroy (but they move and shoot faster).
- Armor tanks: 4 hits to destroy. They flash when hit to show damage stages.
- Player tank: 1 hit to destroy one live

Win: Destroy all tanks in the level. Advance to the next level.

Lose (A): Player runs out of lives (HP reaches 0 three times).

Lose (B): Any bullet (enemy or even player's own) hits the Eagle.

Levels Design — Level 1, 2 & Boss

Each level ramps up in difficulty by introducing more dangerous tank types, more complex map layouts, and tighter AI behaviour. The CSP Map Generator (Module A) must be configured per level to produce the correct difficulty profile.

Level	Name	Enemy Pool (20)	Active Tanks	Map Type	Special Rule
1	Brick Maze	7x Basic + 5x Fast	3 max	Dense brick maze	Fast tanks spawn after 10 kills.
2	Steel Fortress	4x Fast + 3x Armor + 2x power	3 max	Mix of brick and steel	Armor tanks require 4 hits.
Boss	Tank Commander	1x Boss Tank (infinite HP until phase change)	1	Small arena	Boss uses Minimax + Alpha-Beta AI.

Level 1 — Brick Maze

Purpose: Introduce dynamic map changes. Walls get destroyed; paths that were blocked open up. BFS re-planning is tested.

- Map: Dense brick maze. Multiple corridors. Less steel walls. More forest and brick walls
- Enemies: First 7 kills are Basic Tanks (BFS). Final 5 are Fast Tanks (Greedy Best-First).
- AI Behavior: Fast tanks rush aggressively — they do not detour. Watch them drill straight lines through brick.
- CSP Constraint: Eagle must have at least 2 layers of brick protection at level start.
- Key Mechanic: As walls are destroyed mid-level, BFS agents must re-compute paths. Test your re-planning logic here.
- Player Strategy: Use forest tiles to dodge enemy fire and set up ambushes on approaching tanks.

Level 2 — Steel Fortress

Purpose: Introduce Steel Walls as absolute barriers and Armor Tanks that require sustained combat to destroy.

- Map: Mix of brick and steel. Steel walls form partial barriers — forcing tanks to navigate around them.
- Enemies: 4x Fast (Greedy), 3x Armor (A* with defensive behavior), 2x Power Tanks (Utility-Based).
- Armor Tank AI: Uses A* for navigation. On 3rd hit, retreats to find cover behind a steel wall.
- A* cost: steel = infinity (cannot pass), brick = 3 (shoot + wait), empty = 1.
- Player Challenge: You must hit each Armor Tank 4 times. Managing combat against multiple armor tanks is hard.

Boss Level — Tank Commander (Adversarial Mode)

Purpose: A special 1v1 Boss Battle in a small closed arena. The Boss Tank is unique — it uses Minimax with Alpha-Beta Pruning, making it a genuinely strategic opponent.

- Arena: Small 12x12 tile arena. Mixed terrain — some brick, some steel pillars, one water patch.
- Boss HP: 10 hits to destroy. The Boss changes behavior at each HP stage (Phase System).

Phase	HP Remaining	Boss Behaviour	AI Strategy
Phase 1	10 — 7 HP	Aggressive push toward player.	Minimax depth 2. Prioritize closing distance.
Phase 2	6 — 3 HP	Balanced attack + seek cover.	Minimax depth 3 with cover bonus in heuristic.
Phase 3	2-1 HP	Desperate, unpredictable rush.	Minimax depth 4. Maximum aggression. Ignore self-preservation.

- Alpha-Beta Pruning: Optimises the Minimax search so Boss responds within one game tick even at depth 4.
- Boss Evaluation Heuristic: Rewards line-of-sight on player (+50), proximity to player (closer = higher), cover behind steel (+30). Penalises low Boss HP (-40 per missing hit point).

3. Tank Types — Complete Catalogue & Syllabus Mapping

There are distinct tank types in this project:3 regular enemy types that appear across Levels 1-3, and 1 unique Boss Tank for the final Boss Level. Each tank type maps directly to a specific AI syllabus module, a specific search algorithm, and a specific agent architecture. Every tank must be implemented exactly as specified below.

TANK TYPE 1 — Basic Tank

Property	Value	Detail
HP	1 hit	Destroyed in a single bullet.
Speed	Slow (1 tile per 4 ticks)	Slowest tank on the field.
Fire Rate	1 bullet every 3 seconds	Rarely threatens an attentive player.
Agent Model	Simple Reflex Agent	No memory, no planning, pure reaction.
Search Algorithm	BFS (Breadth-First Search)	Used to find shortest-hop path to Eagle.

Basic Tank — Agent Rule Set (Simple Reflex)

Primary Rule: IF player is in the same row OR same column AND no wall tile between them THEN shoot.

Movement Rule: IF path to Eagle exists via BFS THEN follow next BFS step. ELSE turn to a random free direction.

Wall Rule: IF next tile in current direction is Brick THEN shoot to destroy it. THEN resume movement.

Basic Tank — BFS Pathfinding Details

- Goal: Find the shortest-hop path from current position to the Eagle tile.
- Trigger: Re-run BFS (a) at spawn, (b) when current path tile is blocked, (c) every 5 seconds to account for map changes.
- BFS treats all passable tiles (Empty, Forest) as equal cost = 1. It does NOT consider shooting through brick.
- Result: The Basic Tank takes the most direct open-path route, never choosing to destroy walls strategically.
- Implementation: Standard queue-based BFS. Return the next step on the path as the tank's move action.

TANK TYPE 2 — Fast Tank

Property	Value	Detail
HP	1 hit	Destroyed in a single bullet, but hard to hit.
Speed	Fast (1 tile per 2 ticks)	Twice as fast as Basic Tank.
Fire Rate	1 bullet every 1.5 seconds	More frequent shooting than Basic.
Agent Model	Goal-Based Agent	Single goal: reach and destroy the Eagle. Ignores player.
Search Algorithm	Greedy Best-First Search	Rushes toward Eagle using heuristic only — no cost awareness.

Fast Tank — Agent Rule Set (Goal-Based)

Goal: Destroy the Eagle. The Fast Tank does NOT engage the player — it ignores the player completely and rushes the base.

Movement Rule: Always move toward the tile that minimises Manhattan distance to Eagle (Greedy Best-First). Never stop to engage the player.

Wall Rule: IF next tile is Brick THEN shoot it to clear the path. Do NOT detour — push straight through.

Fast Tank — Greedy Best-First Details

- Heuristic h(n): Manhattan distance from current tile to Eagle tile.
- Trigger: Re-compute greedy next step on every tick (no caching needed — it is a single-step decision).
- The Fast Tank does NOT compute a full path — it simply picks the neighbour tile with the lowest h(n) and moves there.
- Consequence: It can get stuck in local minima (e.g., surrounded by walls with one opening behind it).
- This local-minima failure is intentional — it shows students WHY greedy search is not optimal.

TANK TYPE 3 — Armor Tank		
Property	**Value**	**Detail**
HP	4 hits	Flashes on each hit. Sprite changes color each stage.
Speed	Medium (1 tile per 3 ticks)	Slower than Fast but tougher.
Fire Rate	1 bullet every 2 seconds	Moderate shooting pace.
Agent Model	Model-Based Reflex Agent	Maintains internal state (hit counter). Changes behavior when damaged.
Search Algorithm	A* Search	Optimal cost-aware path to Eagle. Recalculates when damaged.

Armor Tank — Agent Rule Set (Model-Based Reflex)

State Variable: hitCount (0 to 3). Tracks how many times this tank has been hit. Persists across ticks.

Rule 1 (0-2 hits): Navigate toward Eagle using A*. If player in line-of-sight, shoot. Continue path.

Rule 2 (3rd hit): RETREAT. Abandon current A* path. Find nearest Steel Wall tile via BFS and move behind it for cover.

Rule 3 (after retreat): Wait 2 seconds behind cover, then re-compute A* path to Eagle and resume attack.

Armor Tank — A* Pathfinding Details

- Heuristic h(n): Manhattan distance to Eagle. Admissible — never overestimates.
- g(n) costs: Empty tile = 1 | Forest tile = 1 | Brick Wall = 3 (shoot + wait penalty) | Steel Wall = infinity (blocked) | Water = infinity (blocked).
- Trigger: Re-run A* at spawn, after retreating to cover, and whenever a wall in the current path is destroyed (map change event).
- Key Insight: A* discovers it is cheaper to shoot through 1 brick wall (cost 3) than walk 6+ empty tiles around (cost 6+). The AI drills through walls strategically.

Why A* here? A* is the gold-standard pathfinding algorithm for cost-aware navigation. The Armor Tank demonstrates A*'s advantage over BFS and Greedy in a cost-heterogeneous environment.

Boss Tank (Tank Commander)		
Property	**Value**	**Detail**
Speed	Variable by phase	Phase 1: slow. Phase 2: medium. Phase 3: fast.
Fire Rate	Variable by phase	Phase 1: 1/2s. Phase 2: 1/1.5s. Phase 3: 1/0.8s.

Agent Model	Adversarial Agent (Minimax)	Simulates player responses up to depth 4.
Search Algorithm	Minimax + Alpha-Beta Pruning	Depth-limited search; depth changes by phase.

Boss Tank — Minimax Decision Process

MAX node: Boss Tank's turn — select action that maximises the evaluation heuristic.

MIN node: Player's simulated response — select action that minimises the Boss's heuristic.

Depth: Phase 1 = depth 2 | Phase 2 = depth 3 | Phase 3 = depth 4. Deeper = smarter but slower.

Alpha-Beta: Prune branches where alpha >= beta. Reduces search from O(b^d) to O(b^{d/2}), enabling real-time response.

Boss Tank — Evaluation Heuristic

Factor	Score	Reason
Player within 3 tiles	+60	Very close — high chance to shoot.
Player in line-of-sight	+50	Can shoot immediately.
Boss adjacent to steel	+30	Has cover from player's bullets.
Player HP missing (per HP)	+20	Player is weakened.
Boss HP missing (per HP)	-40	Boss is losing — de-prioritise.
Player in forest tile	-20	Cannot see player — uncertain.

4. Tank-to-Syllabus Quick Reference

Use this table as your implementation checklist. Every row must be fully implemented for full marks.

Tank	Agent Model	Search Algorithm	Trigger Condition
Basic	Simple Reflex	BFS	Always active
Fast	Goal-Based	Greedy Best-First	Always active
Armor	Model-Based Reflex	A* Search	Behaviour changes on 3rd hit
Boss	Adversarial Minimax	Minimax + Alpha-Beta	Boss Level only

5. Environment & State Space

The game world is a 26x26 grid. The terrain type of each tile determines movement cost, bullet behavior, and AI visibility — the foundation of every search and agent module.

Value	Terrain	Property	A* Cost	AI Note
0	Empty (Road)	Standard movement.	g = 1	Default traversal.
1	Brick Wall	Destructible by bullets.	g = 3	Cheaper to shoot through thin walls than long detours.
2	Steel Wall	Indestructible.	g = infinity	Absolute barrier. Forces detour.
3	Water	Bullets pass; tanks blocked.	g = infinity	Treat as wall for tank pathfinding.
4	Forest	Hides tanks inside it.	g = 1	Enables partial observability.
5	Eagle (Base)	Destroying it = game over.	N/A (goal tile)	Primary target for enemy agents.

6. Module A — Constraint Satisfaction Problems (CSP)

Every level loads a fresh randomly generated map. The CSP generator must produce a map that is always playable — no matter what random choices are made.

Variables, Domains & Constraints

Element	Definition	Detail
Variables	Each of the 676 tiles	X_{i,j} for every tile in the 26x26 grid.
Domain	{0,1,2,3,4,5}	Empty/Brick/Steel/Water/Forest/Eagle.
Constraint 1	Base Safety	Eagle must be surrounded by at least 1 ring of Brick or Steel.
Constraint 2	Reachability	Valid BFS path from every Spawn to Eagle must exist.
Constraint 3	Fairness	No spawn within 10 tiles of the player start.
Constraint 4	Density Balance	No more than 40% of tiles can be wall types.
Constraint 5	Water Placement	Water tiles may not block the only path to Eagle.

Implementation Guidelines

11. Assign terrain types using backtracking search or a CSP solver.
12. Apply forward checking after each tile assignment — prune illegal states early.
13. If any constraint is violated, backtrack and reassign.
14. Run a final BFS reachability check. Reject map if Eagle is unreachable.
15. Per-level configuration: adjust steel density, forest coverage, and wall complexity as level number increases.

7. Module B — Search Algorithms, Agents & Behaviours

Algorithm	Tank	Optimal?	When to Trigger	Expected Behavior	Agent Type	Formal Definition	Key Implementation Requirement
BFS	Basic Tank	Shortest hops (not cost)	Spawn + every 5s + wall destroyed	Steady, predictable path via shortest open route.	Simple Reflex	Action = f(current percept) only. No memory.	IF-THEN rules only. No state variables.
Greedy Best-First	Fast Tank	Not guaranteed	Every tick (single-step)	Rushes forward; may get stuck in local minima.	Goal-Based	Has an explicit goal. Plans actions toward it.	Goal = destroy Eagle. All actions serve this goal.
A*	Armor Tank	Cost-optimal	Spawn + retreat + wall destroyed	Drills through thin brick walls instead of long detours.	Model-Based Reflex	Maintains internal state (hit counter).	hitCount variable. Behavior branches on state.

Feature: Enemy Tank Pathfinding — Three Algorithms, Three Behaviours

The three search algorithms are not interchangeable. Each is assigned to a specific tank type for a specific pedagogical reason. The difference in behavior must be visually observable in gameplay.

> **Key Demonstration:** Place a thin brick wall (1 tile wide) across the direct path and a long detour (6+ empty tiles) around it. BFS takes the detour (ignores cost). A* shoots through the wall (cost 3 < 6). Greedy may get confused. This single test validates all three algorithms.

8. Module C — Adversarial Search

Feature: Boss Battle — Minimax + Alpha-Beta Pruning

The Boss Tank in the Boss Level uses full Minimax search with Alpha-Beta Pruning. It simulates the player's best responses before making each decision, making it a genuinely challenging opponent.

Algorithm Summary

- MAX player: Boss Tank. Maximises evaluation heuristic.
- MIN player: Human Player (simulated). Minimises Boss's heuristic.
- Depth: Varies by Boss phase (2 / 3 / 4). Alpha-Beta makes depth 4 feasible in real-time.
- Branching factor: ~5 actions per turn (Up/Down/Left/Right/Shoot).
- Alpha-Beta reduces tree from O(5^4)=625 nodes to approximately O(5^2)=25 nodes — 25x speedup.

> **Implementation Requirement:** You must measure and report: (a) nodes evaluated without pruning, (b) nodes evaluated with Alpha-Beta, (c) the speedup ratio. Include this in your project report.

9. Deliverables & Grading Rubrics

Deliverables

1. Source Code — all 3 core modules + all required tank types implemented.
2. Project Report — 7-10 pages: design decisions, algorithm analysis, comparisons, results.
3. Demo Video — 3-5 minutes showing each AI module distinctly in gameplay.
4. Oral viva — 10-15minutes viva with code demo and Q/A.

Component	Weight	Key Criteria
Module A — CSP Map Generator	**15%**	Valid maps; all constraints satisfied; backtracking shown.
Module B — Search (BFS + Greedy + A*)	**20%**	All 3 algorithms implemented; behavioural difference visible.
Module C — Adversarial search (Minimax + Alpha-Beta pruning)	**15%**	Boss makes strategic decisions; pruning implemented and measured.
Report & Algorithm Analysis	**10%**	Clear analysis; performance comparisons; node-count data.
Creativity	**10%**	UI, implementation
Oral viva (individual)	**30%**	Marks will be assigned on each individual's understanding and contribution demonstrated in the oral viva.

Good luck — and have fun building your AI agents!